

Project Design Phase
Proposed Solution Template

Date	09 July 2026
Project Name	CUSTOMER REGISTRY
Maximum Marks	2 Marks

Proposed Solution Template:

Project team shall fill the following information in the proposed solution template.

S.No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	Customer service operations often suffer from lost information, delayed responses, and inconsistent service because there is no centralized system to store data, track requests, and route them to the right agents. Customers are forced to repeatedly explain the same issues, while agents juggle multiple channels without shared context.
2.	Idea / Solution description	A centralized Customer Registry web application built using the MERN stack with an MVC architecture. It features role-based access for Customers, Agents, and Admins. The system centralizes customer profiles, tracks complaints with unique IDs, facilitates direct communication, and provides admin dashboards for complaint assignment and performance reporting.
3.	Novelty / Uniqueness	The solution provides a unified platform that integrates profile management, issue ticketing, and full communication history into a single, seamless interface. Its modular MVC architecture ensures flexibility and adaptability to future business or industry requirements.
4.	Social Impact / Customer Satisfaction	Transforms the customer support experience by reducing wait times and frustration. Customers feel valued and in control due to transparent tracking and quicker resolutions. It also empowers support agents by providing clear context, reducing workplace stress, and improving overall service quality.
5.	Business Model (Revenue Model)	A B2B Software-as-a-Service (SaaS) subscription model. Revenue can be generated through tiered pricing based on the number of registered agents (per-seat licensing), advanced administrative reporting features, or the volume of customer records managed.
6.	Scalability of the Solution	The application is built on the MERN stack utilizing the MVC (Model-View-Controller) architectural pattern, ensuring a clear separation of concerns. This allows the

		database (MongoDB), backend logic, and frontend interfaces to scale independently. Its layered MVC structure also makes it straightforward to extend with future integrations, such as SMS or email notification services, as demand grows.
--	--	---